

ADR-003 — FastAPI over Flask and Django

Status: Accepted

Date: 2026-07-06

Context

The project requires a REST API to expose graph analytics functionalities — community detection, recommendation generation, and graph statistics — to any client application. We need to select a Python web framework. The main candidates are FastAPI, Flask, and Django REST Framework. The API must be documented, tested, and containerized within the 8-week internship timeline.

Decision

We will use **FastAPI** as the REST framework.

Reasons

Automatic OpenAPI documentation. FastAPI generates a fully interactive Swagger UI at `/docs` and a ReDoc interface at `/redoc` with zero additional configuration. Flask requires manual integration of `flask-restx` or `flasgger` to achieve the same result. Django REST Framework generates browsable API docs but does not produce OpenAPI 3.0 output out of the box. Given that OpenAPI documentation is an explicit acceptance criterion (CDC section 8), FastAPI eliminates an entire category of manual work.

Request and response validation via Pydantic. FastAPI uses Pydantic models for automatic input validation, type coercion, and error response formatting. This reduces

boilerplate compared to Flask, where validation must be handled manually or through an additional library such as `marshmallow`. It also catches malformed requests at the framework level before they reach business logic.

Performance. FastAPI is built on Starlette and runs on an ASGI server (Uvicorn), making it significantly faster than Flask's WSGI-based architecture under concurrent load. While throughput is not a critical concern for an MVP with a single developer, the ASGI foundation also enables future support for asynchronous Neo4j queries without architectural changes.

Django is oversized for this use case. Django's strengths — ORM, admin interface, authentication system, template engine — are irrelevant for a graph analytics API backed by Neo4j. The added learning overhead and configuration complexity of Django REST Framework are not justified by any feature the project requires.

Lean learning curve. FastAPI requires familiarity with Python type hints, which are already standard in the project's codebase. The additional concepts needed (routers, dependency injection, Pydantic models) can be fully covered in a two-hour learning spike (GDS-15), whereas Django's configuration surface is considerably larger.

Consequences

- The API server runs via `uvicorn app.main:app` inside a Docker container.
- All request and response schemas are defined as Pydantic models in `app/schemas/`, providing a single source of truth for validation and documentation.
- The test suite uses FastAPI's built-in `TestClient` (based on `httpx`), requiring no additional HTTP testing library.
- Asynchronous endpoints can be added in the future using `async def` without changing the framework or server.